AWS Certification Practice — Architecture Blueprint

A detailed, buildable blueprint: services, every connection/link, security, and the exact build order.

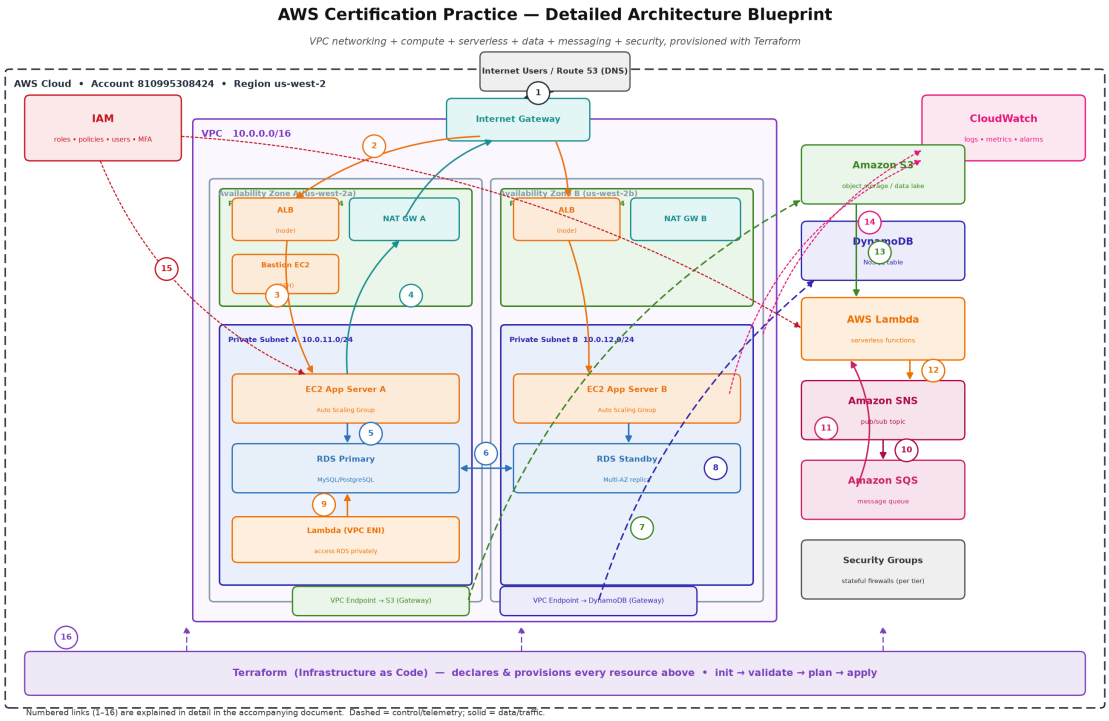

Figure 1 — Full environment. The numbers (1–16) match the connection details in Section 4.

1. What You're Building (and Why)

This is a realistic, exam-aligned AWS environment: a Virtual Private Cloud (VPC) spanning two Availability Zones for high availability, with a public tier (load balancer, NAT, bastion), a private tier (application servers and database), plus serverless and messaging services (Lambda, S3, DynamoDB, SNS, SQS) — all secured with IAM and security groups, and provisioned with Terraform. Building it end-to-end covers the core of the AWS Solutions Architect Associate exam and much of Cloud Practitioner and Developer Associate.

- **Design principles:** Availability: two AZs, Multi-AZ RDS, and an Auto Scaling Group across subnets.
- Security: private subnets for app/DB, security groups per tier, least-privilege IAM, no public database.
- Decoupling: SNS/SQS between producers and consumers; Lambda for event-driven work.

- Cost/security on data access: VPC Gateway Endpoints keep S3/DynamoDB traffic off the internet/NAT.

2. Service Inventory (What to Spin Up)

Service	Role in the architecture	Where it lives
IAM	Users, groups, roles, policies, MFA — controls who/what can do what	Global (account-wide)
VPC	Isolated private network (10.0.0.0/16)	Regional (us-west-2)
Subnets	2 public + 2 private, one pair per AZ	Per Availability Zone
Internet Gateway	The VPC's door to the internet	Attached to the VPC
NAT Gateway	Outbound-only internet for private subnets	Public subnet (1 per AZ)
Route Tables	Direct traffic (public->IGW, private->NAT/endpoints)	Per subnet
Security Groups	Stateful virtual firewalls, one per tier	Attached to ENIs
ALB (Elastic Load Balancer)	Distributes inbound traffic to EC2	Public subnets (multi-AZ)
EC2 + Auto Scaling	Application compute in the private tier	Private subnets
Bastion host (EC2)	Controlled SSH entry to private instances	Public subnet
RDS (Multi-AZ)	Managed relational database (MySQL/PostgreSQL)	Private subnets
S3	Object storage / data lake / static assets	Regional service
DynamoDB	NoSQL key-value table	Regional service
Lambda	Serverless functions (event-driven + VPC-attached)	Regional (optionally in VPC)
SNS	Pub/sub topic (fan-out notifications)	Regional service
SQS	Message queue (buffering, retries, DLQ)	Regional service
VPC Endpoints	Private access to S3 & DynamoDB	In the VPC route tables
CloudWatch	Logs, metrics, alarms, VPC Flow Logs	Regional service
Terraform	Infrastructure as Code — provisions all of the above	Your laptop / CI

3. Network Design (CIDR & Routing Plan)

Resource	CIDR / Setting	Notes
VPC	10.0.0.0/16	65,536 IPs total
Public Subnet A (AZ-a)	10.0.1.0/24	ALB node, NAT GW A, Bastion
Public Subnet B (AZ-b)	10.0.2.0/24	ALB node, NAT GW B
Private Subnet A (AZ-a)	10.0.11.0/24	EC2 App A, RDS primary, Lambda ENI
Private Subnet B (AZ-b)	10.0.12.0/24	EC2 App B, RDS standby
Public Route Table	0.0.0.0/0 -> Internet Gateway	Associated to both public subnets
Private Route Table	0.0.0.0/0 -> NAT Gateway	Plus S3 & DynamoDB endpoint routes

- **Key setting:** Enable 'auto-assign public IP' on public subnets only.
- Turn on VPC Flow Logs -> CloudWatch to observe/allow-deny traffic (great for learning).

4. Every Connection / Link (matches diagram numbers)

This is the heart of the blueprint — how each piece connects. Build them in roughly this dependency order too.

[1] Internet Users / Route 53 -> Internet Gateway

Route 53 (DNS) resolves your domain to the ALB. User traffic (HTTPS 443 / HTTP 80) enters the VPC through the Internet Gateway — the only path in/out to the public internet. Set a Route 53 hosted zone with an ALIAS/A record pointing at the ALB.

[2] Internet Gateway -> Application Load Balancer

The public route table sends 0.0.0.0/0 to the IGW. The ALB has nodes in BOTH public subnets (multi-AZ) and receives inbound 80/443 (ALB-SG allows this from anywhere). The ALB is the public front door; nothing else is publicly exposed.

[3] ALB -> EC2 App Servers (private)

The ALB forwards requests to a Target Group of EC2 instances in the PRIVATE subnets. App-SG allows inbound 80/443 ONLY from ALB-SG (not from the internet). This is the classic public-LB / private-compute pattern.

[4] EC2 (private) -> NAT Gateway -> Internet Gateway (outbound)

Private instances need outbound internet for OS patches, package installs, and calling external APIs. The private route table sends 0.0.0.0/0 to the NAT Gateway (in the public subnet), which forwards to the IGW. Traffic is outbound-only — the internet cannot initiate connections back in.

[5] EC2 App -> RDS

App servers connect to RDS on the DB port (3306 MySQL / 5432 PostgreSQL). DB-SG allows inbound on that port ONLY from App-SG. RDS sits in private subnets with no public accessibility — the database is never exposed.

[6] RDS Primary <-> Standby (Multi-AZ)

RDS Multi-AZ synchronously replicates to a standby in the other AZ and fails over automatically on failure. A DB Subnet Group spanning both private subnets is required to enable this.

[7] Private tier -> S3 via VPC Gateway Endpoint

A Gateway VPC Endpoint for S3 is added as a route in the private route table (an AWS-managed prefix list). EC2/Lambda reach S3 over the AWS backbone instead of the internet/NAT — cheaper and more secure. No public traffic, and you can scope access with an endpoint policy.

[8] Private tier -> DynamoDB via VPC Gateway Endpoint

Same pattern as S3: a Gateway Endpoint for DynamoDB keeps table traffic private and off the NAT/IGW path.

[9] Lambda (in VPC) -> RDS

A Lambda function attached to the private subnets gets an ENI and connects to RDS using Lambda-SG (allowed by DB-SG). Use this for serverless DB access without running an EC2 server. Note: VPC-attached Lambdas reach S3/DynamoDB via the endpoints, and the internet only via NAT.

[10] SNS -> SQS (fan-out)

An SNS topic delivers each published message to one or more subscribed SQS queues. This decouples producers from consumers and lets multiple systems process the same event independently (fan-out).

[11] SQS -> Lambda (event source mapping)

Lambda polls the SQS queue via an event source mapping and processes messages in batches. SQS buffers spikes and enables automatic retries; failed messages go to a Dead-Letter Queue (DLQ). This is the resilient async processing pattern.

[12] Lambda -> SNS (publish)

Lambda publishes notifications/events to an SNS topic; subscribers (email, SMS, other Lambdas, or SQS queues) receive them. Used for alerts and event broadcasting.

[13] S3 event -> Lambda (ObjectCreated trigger)

Uploading an object to S3 fires an event that triggers a Lambda (event-driven ingestion). This is the exact pattern in your order-processor project (S3 -> Lambda -> DynamoDB). The function needs s3:GetObject.

[14] Everything -> CloudWatch

Lambda logs, EC2 logs (via the CloudWatch agent), RDS metrics, ALB metrics, and VPC Flow Logs all flow to CloudWatch. Create alarms (e.g., high CPU, RDS connections, SQS queue depth) and dashboards. Central observability is an exam favorite.

[15] IAM Roles -> EC2 & Lambda

EC2 instances use an Instance Profile (IAM role) to call AWS APIs (e.g., read S3) WITHOUT hardcoded keys. Lambda uses an Execution Role. Both are least-privilege — scoped to specific buckets/tables/queues. Users/groups (admin, developer) authenticate with MFA. IAM never grants access to the root user's power.

[16] Terraform -> provisions everything

A single Terraform codebase declares the VPC, subnets, route tables, IGW, NAT, endpoints, security groups, EC2/ASG, ALB, RDS, S3, DynamoDB, SNS, SQS, Lambda, IAM, and CloudWatch — reproducibly. init -> validate -> plan -> apply, and destroy to tear it all down.

5. Security Groups (Firewall Rules per Tier)

Security groups are stateful (return traffic is auto-allowed). Reference other security groups as the source, not IP ranges, wherever possible — that's the secure, exam-correct approach.

Security Group	Inbound (allow)	Source	Outbound
ALB-SG	80, 443	0.0.0.0/0 (internet)	to App-SG
App-SG (EC2)	80, 443	ALB-SG	all (via NAT) + to DB-SG
	22 (SSH)	Bastion-SG	
Bastion-SG	22 (SSH)	YOUR_IP/32 only	to App-SG
DB-SG (RDS)	3306 or 5432	App-SG and Lambda-SG	(none needed)
Lambda-SG	(none inbound)	-	to DB-SG + endpoints

6. IAM Roles & Policies (Least Privilege)

Identity / Role	Trusts / used by	Key permissions (scoped)
EC2 Instance Role	ec2.amazonaws.com	s3:Get/Put (app bucket), CloudWatch Logs, SSM
Lambda Execution Role	lambda.amazonaws.com	DynamoDB, S3 (scoped), SQS receive/delete, SNS publish, RDS connect, Logs
Admin user (you)	human + MFA	AdministratorAccess (day-to-day, not root)
Developer group	human + MFA	scoped dev permissions

- **Rule:** Enable MFA on the root user and on any human user; never use root keys for daily work.

7. Build Order (Dependency-Ordered Checklist)

Create resources in this order (each depends on the ones above it). This is also a good order to write the Terraform modules.

1. **1.** IAM — roles, policies, groups, MFA.
2. **2.** VPC (10.0.0.0/16).
3. **3.** Subnets — 2 public + 2 private across two AZs.
4. **4.** Internet Gateway — create and attach to the VPC.
5. **5.** Public route table — 0.0.0.0/0 -> IGW; associate public subnets.
6. **6.** NAT Gateway (+ Elastic IP) in a public subnet.
7. **7.** Private route table — 0.0.0.0/0 -> NAT; associate private subnets.
8. **8.** VPC Gateway Endpoints — S3 and DynamoDB (add to private route table).
9. **9.** Security Groups — ALB, App, Bastion, DB, Lambda.
10. **10.** RDS — DB subnet group + Multi-AZ instance in private subnets.
11. **11.** EC2 — launch template + Auto Scaling Group (private) + Bastion (public).
12. **12.** ALB — target group + listener (80/443) -> EC2 target group.
13. **13.** S3 bucket + DynamoDB table.
14. **14.** SNS topic + SQS queue (+ DLQ) + SNS->SQS subscription.
15. **15.** Lambda functions + event source mappings (SQS) + S3 notification + VPC config.
16. **16.** CloudWatch — log groups, alarms, dashboards, VPC Flow Logs.

7.1 The Same Order, A to Z (with dependencies)

A more granular view — create each in this order; the note says what it depends on.

- **A.** Secure root + create admin IAM user with MFA (never build on root).
- **B.** Configure AWS CLI (region us-west-2); verify aws sts get-caller-identity.
- **C.** Set a Billing Budget alarm (before anything billable).
- **D.** IAM policies (customer-managed). [depends on: nothing]
- **E.** IAM roles — EC2 instance role + Lambda execution role. [depends on: D]
- **F.** VPC (10.0.0.0/16).
- **G.** Subnets — 2 public + 2 private across two AZs. [depends on: F]
- **H.** Internet Gateway — create AND attach to VPC. [depends on: F]
- **I.** Elastic IP — for the NAT. [needed before J]
- **J.** NAT Gateway — in a public subnet, using the EIP. [depends on: G, I]
- **K.** Route tables — public (->IGW) + private (->NAT), associate subnets. [depends on: H, J, G]
- **L.** VPC Gateway Endpoints — S3 + DynamoDB (into private route table). [depends on: K]
- **M.** Security groups — ALB/App/Bastion/DB/Lambda; create empty, then add rules. [depends on: F]
- **N.** S3 bucket.
- **O.** DynamoDB table.
- **P.** RDS — DB subnet group, then Multi-AZ instance (private, DB-SG). [depends on: G, M]
- **Q.** EC2 Launch Template — App-SG + IAM instance profile. [depends on: M, E]

- **R.** Target Group — for the ALB. [depends on: F]
- **S.** Application Load Balancer + Listener -> Target Group. [depends on: G, M, R]
- **T.** Auto Scaling Group — private subnets, attach to Target Group. [depends on: Q, R]
- **U.** Bastion host (EC2) — public subnet, Bastion-SG (SSH from your IP). [optional]
- **V.** SQS queue + Dead-Letter Queue. [before X, Y]
- **W.** SNS topic.
- **X.** SNS -> SQS subscription + queue policy. [depends on: V, W]
- **Y.** Lambda functions + triggers (SQS mapping, S3 notification, SNS publish). [depends on: E,N,O,P,V,W]
- **Z.** CloudWatch — log groups, alarms, dashboards, VPC Flow Logs. [last — monitors all]

Logic in one line: Identity -> Network -> Firewalls -> Data -> Compute -> Messaging -> Serverless -> Monitoring.

- **Gotcha 1:** The IGW must be ATTACHED to the VPC (creating it isn't enough).
- **Gotcha 2:** The NAT needs an Elastic IP and lives in a public subnet.
- **Gotcha 3:** Security groups reference each other — create empty, then add rules.

8. Suggested Terraform Module Layout

Extend your existing terraform-aws-practice project with these modules (same enable_* toggle pattern):

- modules/vpc/ — VPC, subnets, IGW, NAT, route tables, endpoints, Flow Logs.
- modules/security_groups/ — the five SGs with cross-references.
- modules/ec2_asg/ — launch template, Auto Scaling Group, ALB, target group, listener.
- modules/rds/ — subnet group + Multi-AZ instance + DB-SG wiring.
- modules/messaging/ — SNS topic, SQS queue, DLQ, subscription.
- modules/lambda_function/ + modules/dynamodb_table/ + modules/s3_bucket/ — you already have these.
- modules/iam/ — roles/policies for EC2 and Lambda.

9. Cost Awareness & Certification Coverage

- **Cost:** NAT Gateway and RDS Multi-AZ are the main paid items (NAT ~$0.045/hr + data; RDS db.t3.micro is Free Tier single-AZ, Multi-AZ is not). Practice with them, then terraform destroy to stop charges.
- EC2 t3.micro, S3, DynamoDB, Lambda, SQS, SNS have generous Free Tiers.
- Set a Billing Budget alarm (e.g., alert over $5) before you start.
- **Cert coverage:** This architecture maps directly to Solutions Architect Associate (SAA-C03): VPC, subnets, IGW/NAT, route tables, security groups, ALB, EC2/ASG, RDS Multi-AZ, S3, DynamoDB, Lambda, SQS/SNS, IAM, VPC endpoints, CloudWatch. It also covers Cloud Practitioner and much of Developer Associate.